

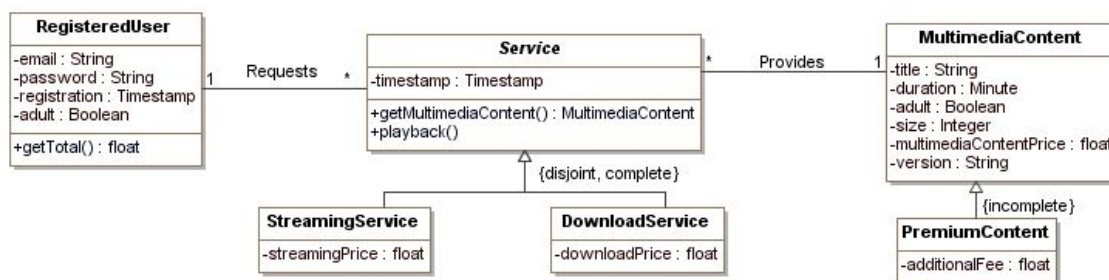
Análisis y Diseño con Patrones

Práctica 2: Principios de asignación de responsabilidades y de diseño

Para poder elaborar esta Práctica partiremos del diagrama de clases de la Práctica 1 con algunas modificaciones.

Recordad que nos han contratado para desarrollar un software para gestionar el streaming de películas en línea. Como la solución publicada para la práctica 1 tenía diversas variantes, mostramos aquí el diseño de partida, en el que hemos hecho algunos cambios menores (comentados al pie del diagrama).

El modelo del dominio de partida es este:



Los cambios hechos respecto al modelo de dominio de la práctica 1 son los siguientes:

- Hemos cambiado el precio de los servicios que piden los usuarios. Los contenidos multimedia tendrán un precio que se verá incrementado con el `additionalFee` para el caso de los contenidos premium. Además habrá un precio de streaming y otro de download en función del tipo de servicio que los usuarios soliciten. Esto significa que diferentes servicios pueden tener precios diferentes en función de una serie de condiciones que no tenemos registradas en el sistema (por ejemplo, si el servicio es nocturno, si el cliente ha pedido un servicio, etc ...).
- Hemos añadido la operación `playback()` en la clase `Service` que inicia la reproducción/ descarga del contenido que proporciona el servei.
- Hemos añadido la versión actual del contenido multimedia.

Ejercicio 1 (20%)

Suponemos que la empresa que se encarga de comercializar las películas en streaming (NetUOC) quiere registrar en el sistema el % de beneficio que cobrará a todo servicio que los usuarios registrados pidan. Se pide:

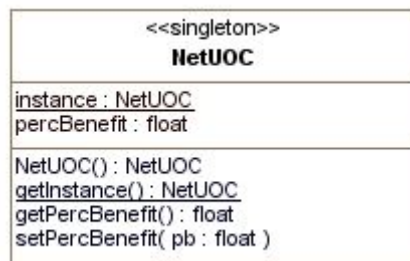
- ¿Qué patrón de diseño es adecuado para registrar este % de beneficio?
- Realizad el diagrama de clases resultante de aplicar este patrón (sólo la parte de la aplicación del patrón)
- Realizad el pseudocódigo del diagrama de clases que proponéis en el apartado anterior.

Nota:

- El % de beneficio será utilizado más adelante en la práctica para calcular el precio de cada servicio. No hace falta que en este ejercicio tengáis en cuenta este cálculo.

Solución:

- Aplicaremos el patrón *Singleton* a la clase *NetUOC* para mantener una única instancia de esta clase que tenga definido el % de beneficio que quiere obtener la empresa para cada servicio que sea proporcionado. Este % de beneficio se guardará una única vez para todos los servicios.
- El diagrama de clases resultante del patrón tendrá definida una única clase.



- ```
public class NetUOC {
 private static NetUOC instance = null;
 private float percBenefit;
 private NetUOC() {}
 public static NetUOC getInstance() {
 if (instance == null) {
 instance = new NetUOC();
 }
 }
}
```

```

 }
 return NetUOC;
 }
 public float getPercBenefit() {
 return percBenefit;
 }
 public float setPercBenefit(pb) {
 percBenefit = pb;
 }
}

```

## Ejercicio 2 (20%)

Suponed que NetUOC quiere calcular el precio de cada servicio con la operación *getPrice(): Float* definida en la clase *Service*. Si el servicio es un servicio de streaming y proporciona un contenido multimedia no premium entonces el precio será:  $\text{streamingPrice} + \text{multimediaContentPrice} + \% \text{benefici (de NetUOC)}$ . Si el servicio es un servicio de streaming y proporciona un contenido premium entonces el precio será:  $\text{streamingPrice} + \text{multimediaContentPrice} + \text{additionalFee} + \% \text{beneficio (de NetUOC)}$ . Si el servicio es un servicio de download y proporciona un contenido multimedia no premium entonces el precio será:  $\text{downloadPrice} + \text{multimediaContentPrice} + \% \text{beneficio (de NetUOC)}$ . Si el servicio es un servicio de download y proporciona un contenido premium content entonces el precio será:  $\text{downloadPrice} + \text{multimediaContentPrice} + \text{additionalFee} + \% \text{beneficio (de NetUOC)}$ . Se pide:

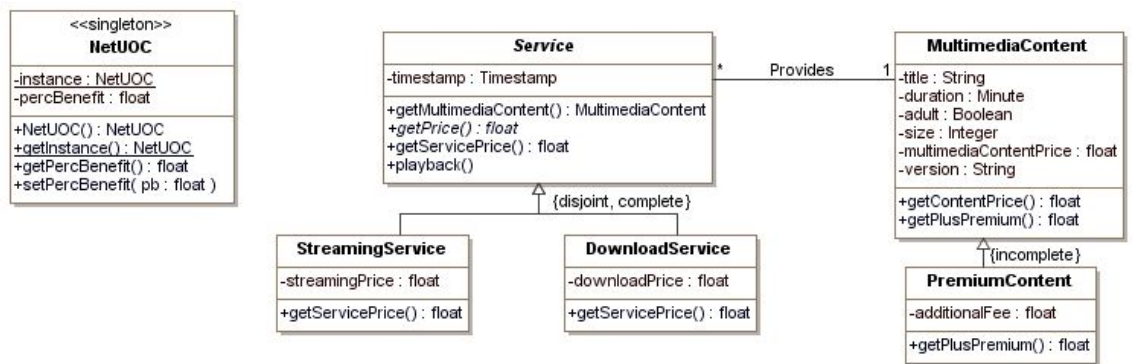
- ¿Qué patrón de diseño es adecuado aplicar en esta situación?
- Realizad el diagrama de clases con las operaciones resultantes de aplicar este patrón (podéis proporcionar sólo la parte de la aplicación del patrón)
- Realizad el pseudocódigo o el diagrama de secuencia de la operación *getPrice():Float* y de todas las operaciones que sean invocadas desde esta operación.

Nota:

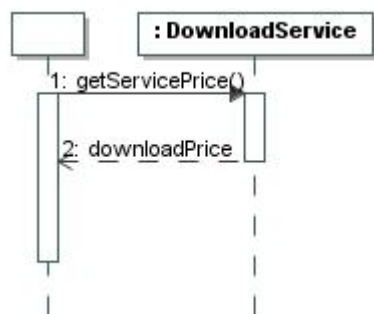
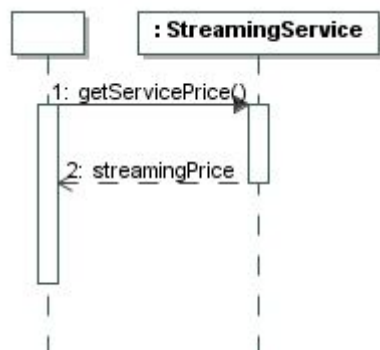
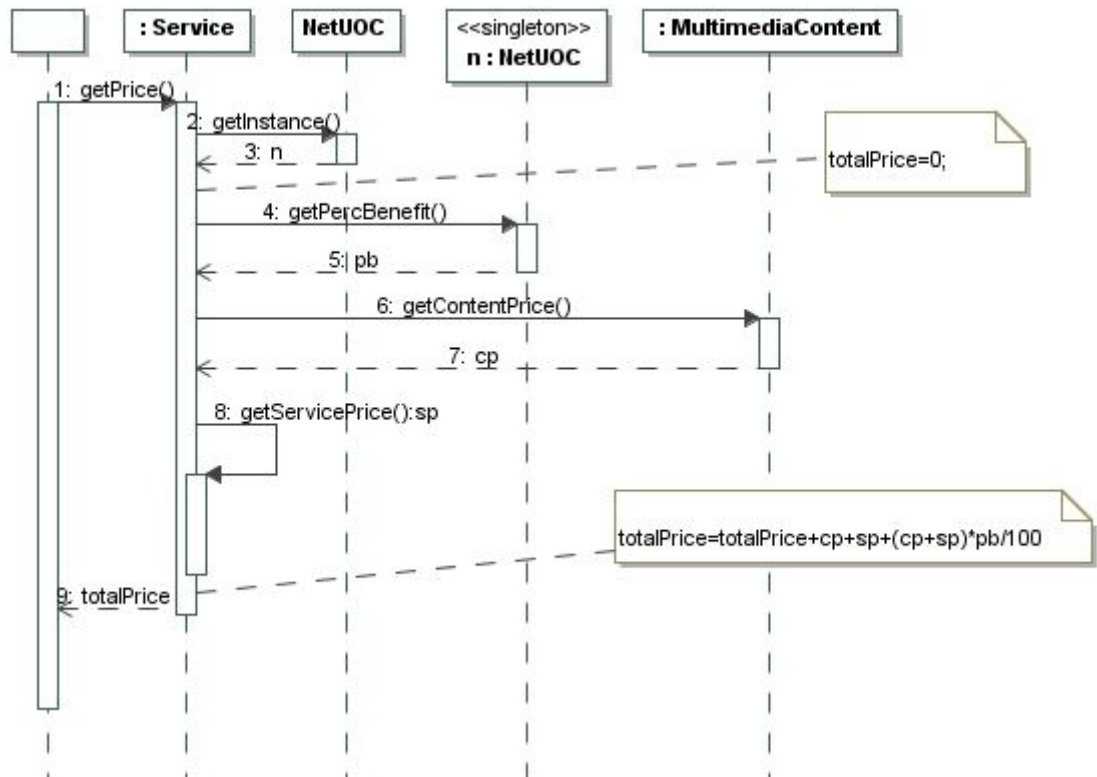
- Se valorará especialmente que en la solución propuesta no haya repetición de código.

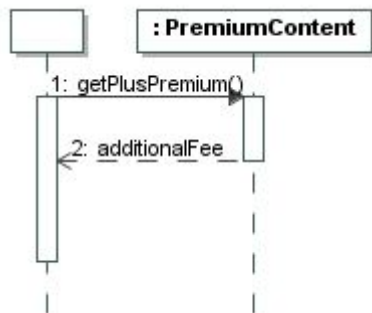
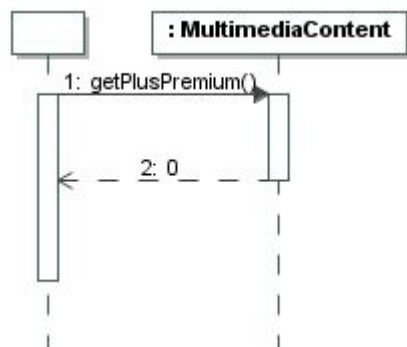
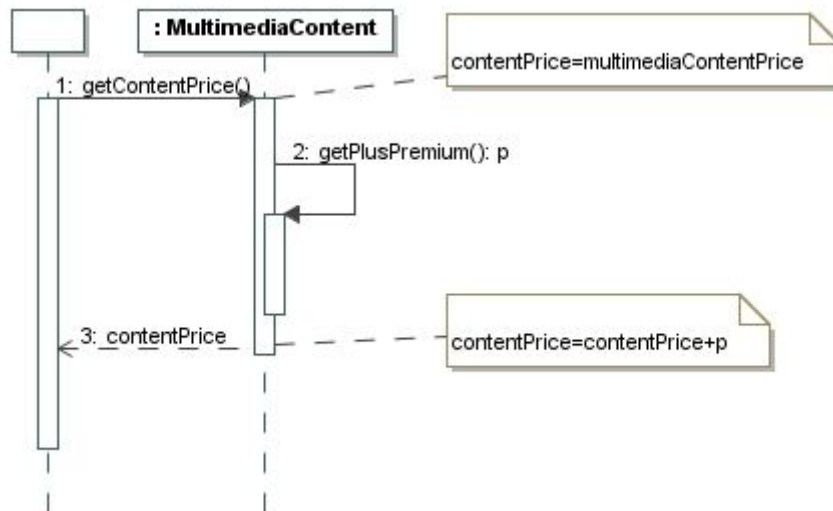
## Solución:

- a) Aplicaremos el patrón *método Plantilla* dos veces, a el método que calcula el precio de un MultimediaContent y al método *getPrice()* que calculará el precio de un servicio. En el primer método la parte común corresponde a la obtención del multimediaContentPrice y la parte específica a la obtención del additionalFee cuando el MultimediaContent es un PremiumContent. En el segundo método la parte comuna corresponde al cálculo del precio del multimedia content más el % beneficio de NetUOC y la parte específica corresponde a la obtención del streamingPrice o downloadPrice (en función del tipo de servicio). Com se puede comprobar en el apartado c), esta solución evita la repetición de código ya que la parte común de las operaciones se define una única vez.
- b) El diagrama de clases resultante de la aplicación del patrón método plantilla en las operaciones *getPrice()* y *getContentPrice()* se puede encontrar a continuación.



c)





## Ejercicio 3 (20%)

Suponed que queremos diseñar la operación  
`createDownloadServiceWithPremiumContent(ts: Timestamp, dlPrice: float,  
pc:PremiumContent): Integer` de la clase *RegisteredUser*. Esta operación que crea un  
servicio de download para el usuario registrado con un contenido premium tiene las  
siguientes precondiciones (condiciones que se satisfacen antes de invocar a la  
operación):

- No existe un objeto *DownloadService* con los datos de los parámetros de la operación.
- El parámetro *downloadPrice* es más grande que 0.

La operación tiene que hacer lo siguiente:

- Crear un objeto *DownloadService* *ds* y asignarle el *downloadPrice* *dIPrice* y el *timestamp* *ts* y el *PremiumContent* *pc*.
- Guardar al registered user el objeto *ds* creado.
- Calcular i retornar el número total de servicios de download con contenido premium que tiene el usuario registrado.

Se pide:

- ¿A qué clase corresponde la responsabilidad de implementar el cálculo del número total de servicios de download con contenido premium que tiene el usuario registrado? Justifica la respuesta.
- ¿Qué patrones de diseño son adecuados para aplicar en esta situación?
- Realizad el pseudocódigo o el diagrama de secuencia de la operación *createDownloadServiceWithPremiumContent* y de todas las operaciones que sea invocadas desde esta operación.

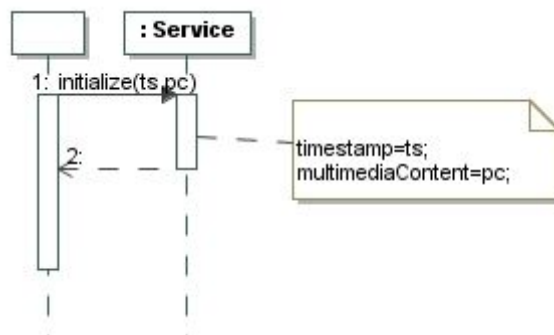
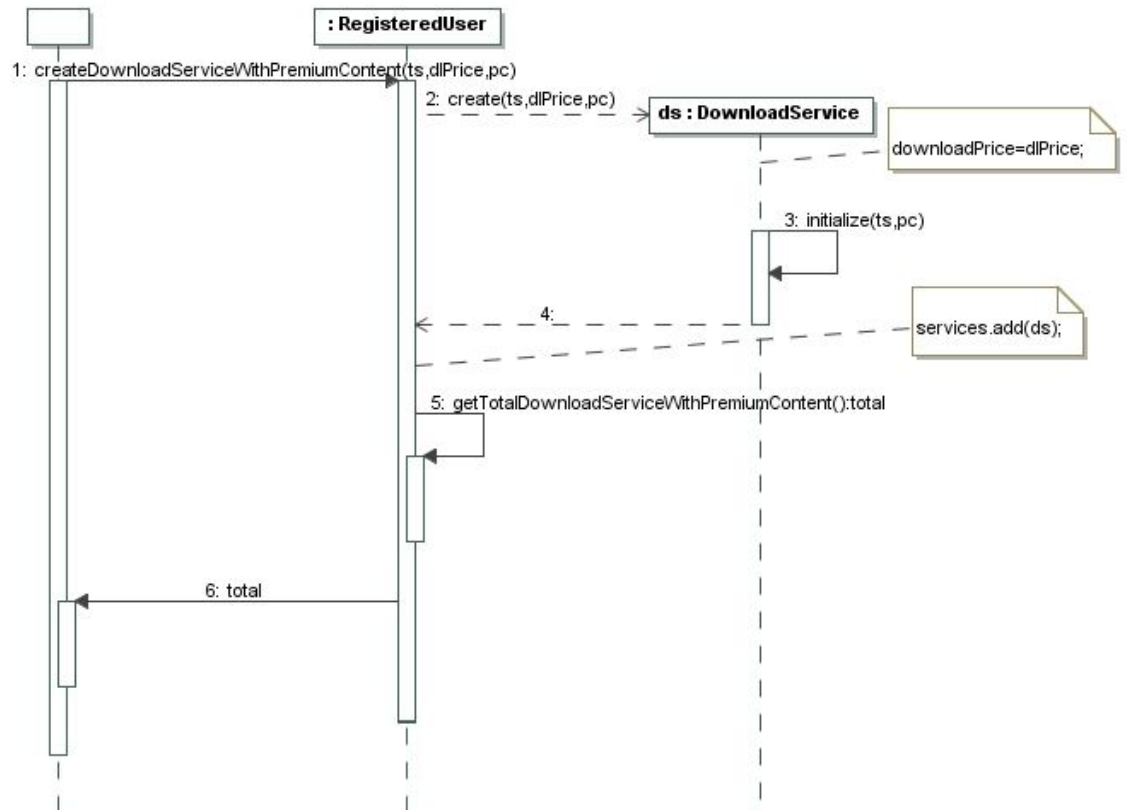
Nota:

- Nos dicen que el cálculo del número total de servicios de download con contenido premium que tiene un usuario registrado también es necesario para otras operaciones del sistema que estamos desarrollando. Se valorará especialmente que vuestra solución sea reusable.

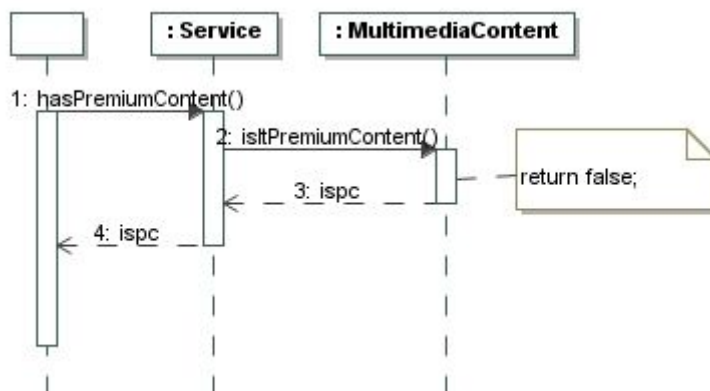
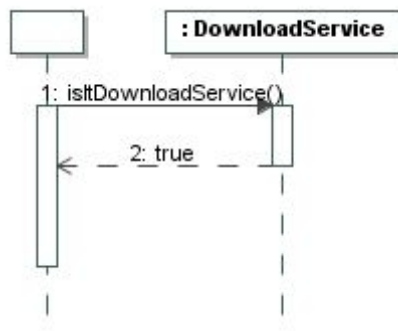
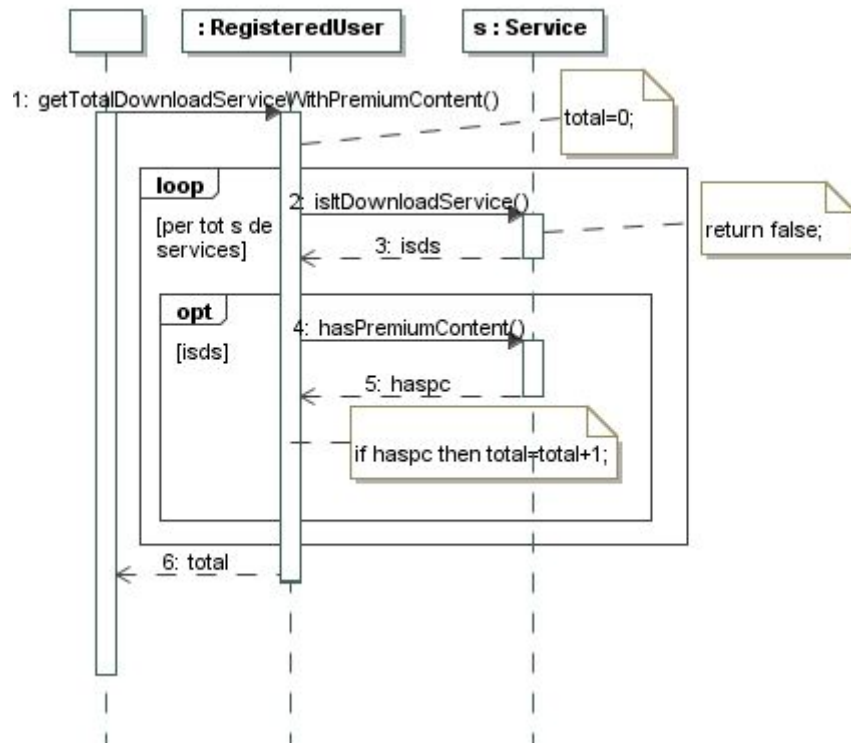
## Solución:

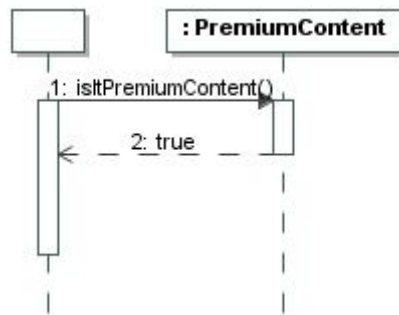
- La clase que contiene la información necesaria para poder realizar este cálculo es la clase *RegisteredUser*. Esta clase tiene definidos los servicios que el usuario registrado ha pedido y, por tanto, aplicando el patrón *Experto* es la clase donde se tiene que definir este cálculo.
- Tenemos que aplicar el patrón *Creador* y asignar a la clase *RegisteredUser* la responsabilidad de crear un servicio de download ya que esta clase contiene los objetos servicios.
- A continuación se muestra el diagrama de secuencia de la operación *createDownloadServiceWithPremiumContent*. Notad que se ha creado una operación para realizar el cálculo del número total de servicios de download

con contenido premium que tiene un usuario registrado para que esta operación pueda ser reusada en otras operaciones del sistema.









## Ejercicio 4 (20%)

NetUOC nos pide hacer una modificación en los requisitos del sistema que estamos desarrollando. Hasta ahora los usuarios registrados podían reproducir/descargar las veces que quisieran los contenidos asociados a sus servicios utilizando la operación *playback()* de la clase *Service*. A partir de ahora se quiere que determinados servicios tengan un límite de 4 reproducciones/descargas de sus contenidos y otros se puedan reproducir/descargar tantas veces como el usuario desee (la decisión de si hay un límite de reproducciones se toma en el momento de la creación del servicio y su asignación al usuario registrado, y queda fuera del alcance de este ejercicio). Debe considerarse que no puede modificar la operación *playback()* de la clase *Service*. Se pide:

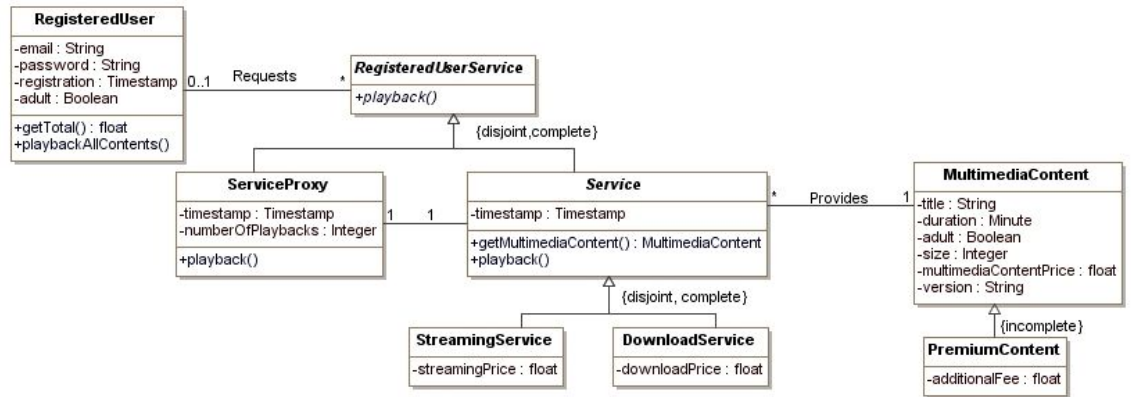
- ¿Qué patrón de diseño es adecuado aplicar en esta situación?
- Realizad el diagrama de clases con las operaciones resultantes de aplicar este patrón (podéis proporcionar sólo la parte de la aplicación del patrón)
- Realizad el pseudocódigo o el diagrama de secuencia de la operación *playbackAllContents()* de la clase *RegisteredUser* que reproduzca/descargue todos los contenidos asociados a sus servicios y de todas las operaciones que sean invocadas desde esta operación.

Nota:

- Si un usuario registrado intenta reproducir/descargar un contenido que tiene un límite de reproducciones/descargas y ya ha llegado al límite, no se hará nada. En caso contrario, se reproducirá/descargará el contenido mediante la invocación de la operación *playback()*.

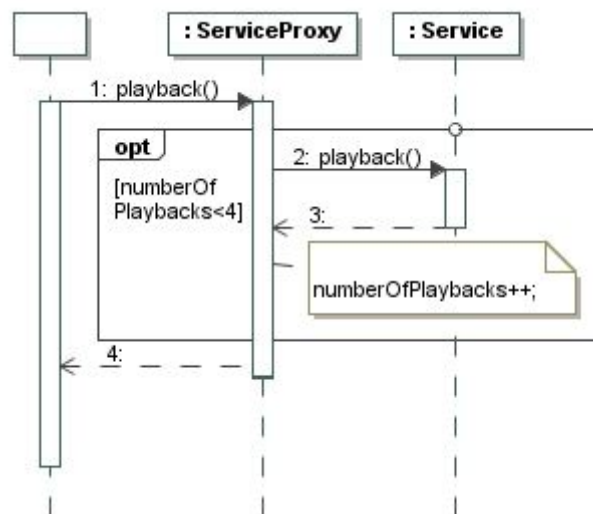
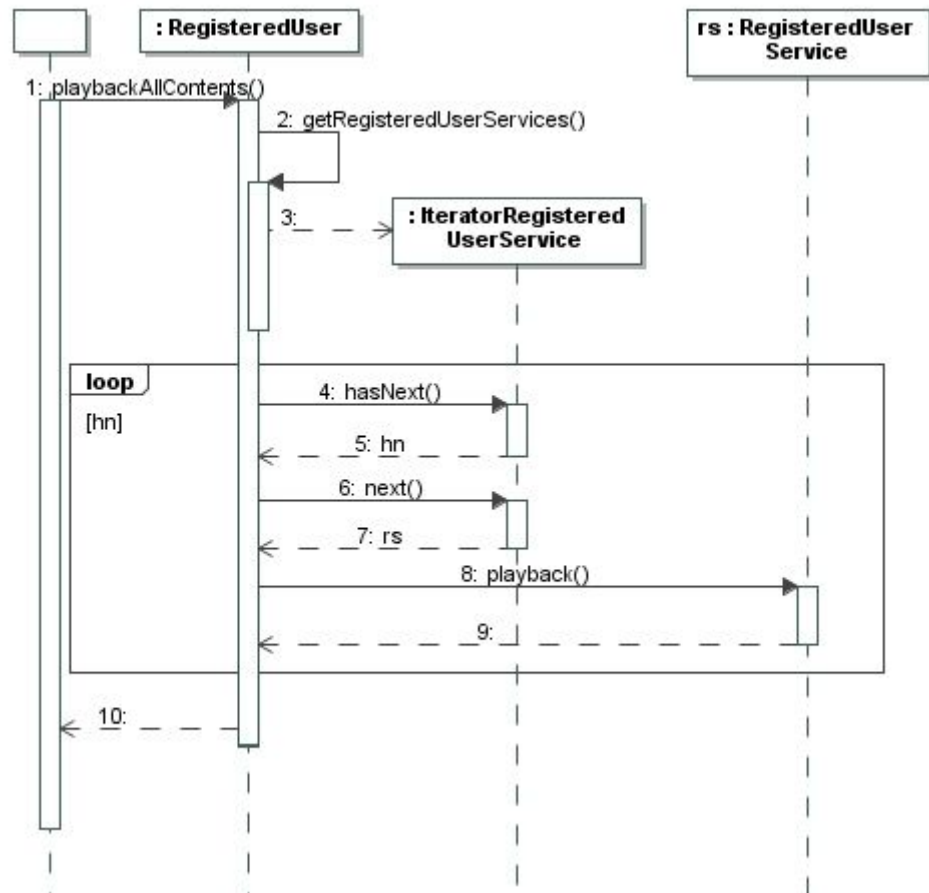
**Solució:**

- a) El patrón principal que usaremos para tratar esta situación es el patrón *Representante*. Para los servicios que tengan un límite de descargas, el representante controlará si se ha llegado al límite de reproducción/descarga (invocación de la operación *playback()*). Para el resto de servicios, la reproducción/descarga se hará invocando directamente a la operación *playback()*.
- b) A continuación disponéis de la parte del diagrama de clases que hemos modificado para aplicar el patrón *Representante*.



Nota: Otra solución podría ser dejar el atributo `timestamp` en la clase *RegisteredUser* y no definirlo en las subclases.

- c) A continuación se muestra el diagrama de secuencia de la operación *playbackAllContents()*.



## Ejercicio 5 (20%)

Suponed que NetUOC quiere ofrecer a sus usuarios registrados la posibilidad de suscribirse a diferentes contenidos para ser notificados via mail cuando llegue una nueva versión del mismo. Cada vez que haya una nueva versión de un contenido, los

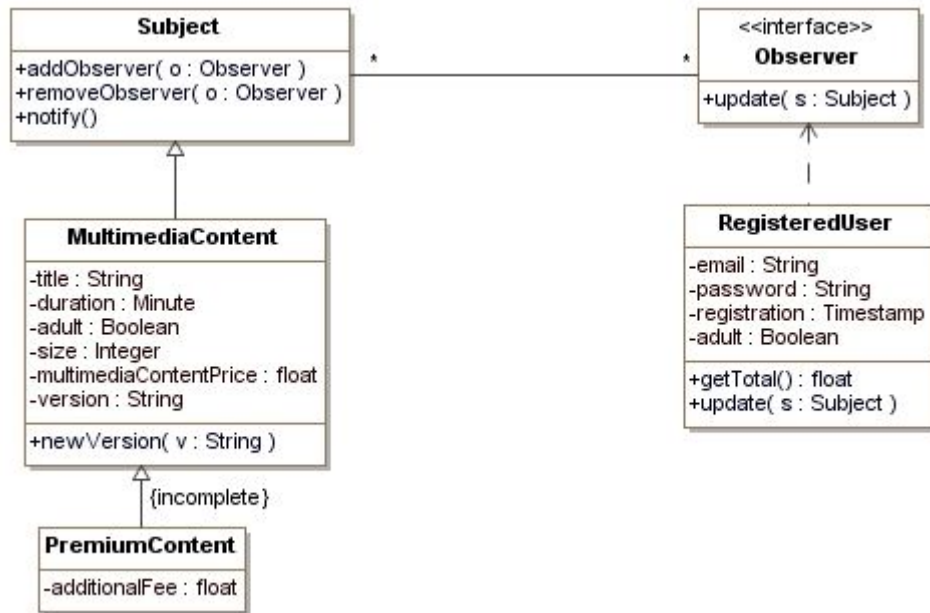
usuarios registrados que estén suscritos recibirán un mail con el texto siguiente: “The new version ”+ version + “ of the multimedia content “ + title + “ is available.”. Podéis suponer la existencia de la operación `sendMail(email: String, text: String)` que puede ser invocada desde donde queráis y que envía el mail con el texto indicado en el parámetro a la dirección de email indicada en el parámetro. Se pide:

- a) ¿Qué patrón de diseño es adecuado aplicar en esta situación?
- b) Realizad el diagrama de clases con las operaciones resultantes de aplicar este patrón (podéis proporcionar sólo la parte de la aplicación del patrón)
- c) Realizad el pseudocódigo o el diagrama de secuencia de la operación `newVersion(v: String)` de la clase `MultimediaContent` que actualiza la nueva versión `v` del contenido y notifica a sus suscritos esta actualización. Realizad también el diagrama de secuencia de todas las operaciones que sean invocadas desde esta operación.

Nota: No queremos que la solución acople los contenidos multimedia con los usuarios registrados.

## Solución:

- a) El patrón principal que usaremos para tratar esta situación es el patrón *Observador*. Cuando llegue una nueva versión de un contenido la operación *notificar del patrón* se encargará de actualizar todos los observadores (*RegisteredUser*) que estén asociados con el contenido. La operación de actualizar será la encargada de enviar el mail.
- b) A continuación disponéis de la parte del diagrama de clases donde hemos aplicado el patrón *Observador*.



c) A continuación se muestra el diagrama de secuencia de la operación *newVersion(v: String)*.

